

The Case for L^AT_EX

Henry Ginn

May 3, 2026

Contents

1	Introduction	2
2	Formatting	3
2.1	Consistency	3
2.2	Word vs L ^A T _E X	3
2.3	Advanced Features	3
3	Technical Writing	4
3.1	Mathematics	4
3.2	Images	4
3.3	Plots and Diagrams	4
3.4	Other	5
4	Version Control	6
4.1	Current Practice	6
4.2	Resolution with L ^A T _E X and Git	6
4.3	Comments and High-Level Changes	8
4.4	Other Features of Git	9
4.5	Implementation	9
5	Ease of Use	10
5.1	Overview	10
5.2	Most Complicated Example	10
6	Failures of Word	11
6.1	Attaching PDF Documents in Appendices	11
6.2	Version Control	11
6.3	Changing Template	11
6.4	Assorted Complaints of Word	12
7	Conclusion	13

1 Introduction

This document gives a brief overview of what $\text{\LaTeX}$ (pronounced “Lay-tek”) is and highlights some of the advantages it has over Microsoft Word. In a sentence, $\text{\LaTeX}$ is a well-established piece of typesetting software for producing technical documents where the input is given in plain text which is then compiled into a PDF. I believe that using $\text{\LaTeX}$ will improve upon several weaknesses in our document production and document control processes and is in line with our digital philosophy of using technology to improve our workflow. It is understandable why one may be wary of any potential barriers to those less comfortable with technology, but the payoff is more than worth it. The main advantages are as follows:

- No need to worry about formatting - template is followed automatically.
- Equations, cross-references, and citations are far easier than in Word.
- Compatible with text-based version control systems such as Git.
- The $\text{\LaTeX}$ compiler knows how to typeset better than you do so documents look professional.
- Less buggy than Word.
- Vector image compatible and are given by reference.

Below I have included the code used to create this section as an example of what $\text{\LaTeX}$ may look like to a typical user¹. Pressing the compile button converts this into a PDF.

```
\section{Introduction}
\label{sec:Introduction}
```

This document gives a brief overview of what $\text{\LaTeX}$ (pronounced “Lay-tek”) is and highlights some of the advantages it has over Microsoft Word. In a sentence, $\text{\LaTeX}$ is a well-established piece of typesetting software for producing technical documents where the input is given in plain text which is then compiled into a PDF. I believe that using $\text{\LaTeX}$ will improve upon several weaknesses in our document production and document control processes and is in line with our digital philosophy of using technology to improve our workflow. It is understandable why one may be wary of any potential barriers to those less comfortable with technology, but the payoff is more than worth it. The main advantages are as follows:

```
\begin{itemize}
  \item No need to worry about formatting - template is followed automatically.
  \item Equations, cross-references, and citations are far easier than in Word.
  \item Compatible with text-based version control systems such as Git.
  \item The  $\text{\LaTeX}$  compiler knows how to typeset better than you do so documents
look professional.
  \item Less buggy than Word.
  \item Vector image compatible and are given by reference.
\end{itemize}
```

Below I have included the code used to create this section as an example of what $\text{\LaTeX}$ may look like to a typical user\footnote{Note that this is done by reference to a file so it will update whenever this introduction is changed. This “single source of truth” functionality is common in $\text{\LaTeX}$.}. Pressing the compile button converts this into a PDF.

¹Note that this is done by reference to a file so it will update whenever this introduction is changed. This “single source of truth” functionality is common in $\text{\LaTeX}$.

2 Formatting

2.1 Consistency

A document with poor or inconsistent formatting makes WSP look unprofessional, but fixing formatting is not something we should need to waste our time or client's money on. $\text{\LaTeX}$ ensures consistency by defining it in one place only and applying it globally across the document². Once a template is defined, the formatting is abstracted away from the author so they can focus solely on the content.

Some may argue that Word can do this through the styles feature, but it is easy to break the template in Word. For example, many times I have had section numbering disappear even though the numbering is part of the section header style and where reapplying the style did not fix it. In my experience, styles do not work reliably in tables either and properties such as spacing after paragraph need to be manually corrected around tables and figures. The user should not need to put any work into ensuring that the template is followed - the software should handle this for us.

Creating a template can be quite complicated, although this would only need to be defined once at the start of the project and users would not need such proficiency with $\text{\LaTeX}$. I am giving some examples here only to illustrate how formatting can be defined in such a text-based style. In the document preamble (code that is not included in the document itself and where the template is defined), you could use the following to change the section numbering. While ugly and not something you would do in practice, this would make this subsection number look like II:A.

```
\renewcommand{\thesection}{\Large{\Roman{section}}}  
\renewcommand{\thesubsection}{\thesection:\tiny{\Alph{subsection}}}
```

2.2 Word vs $\text{\LaTeX}$

Often I find Word does weird things that no one understands and there is no good fix. For example, I have had tables where the boldness of text would not save and the typical solution to a problem like this is to recreate the whole table from scratch or copy and paste everything into a new document. I have seen colleagues input tables as images rather than deal with Word's formatting, a veritable crime against typesetting that looks extremely unprofessional (this also means you cannot search or easily copy text in the table). Pasting tables from other documents that use a different template is also a hassle, yet with $\text{\LaTeX}$ it is a simple copy and paste.

The exact positioning of elements is not determined by the user in $\text{\LaTeX}$. Instead this is done by the compiler which was designed with best typesetting practice in mind, meaning documents look far more professional than in Word. For example, Word splits tables across pages by default, but $\text{\LaTeX}$ tries hard to avoid doing this where it can. I have read textbooks made with $\text{\LaTeX}$ using out-of-the-box settings, yet work done with Word would need considerable tweaking to be brought up to a publishable standard.

2.3 Advanced Features

$\text{\LaTeX}$ also allows for more advanced features than Word and gives a lot more control. As simple examples, it is possible to adjust how much weight it should give towards avoiding widows or exactly how much space you want between two elements. For a more complicated example, WSP recently changed the template used for CVs. With $\text{\LaTeX}$ this could have been done with a simple script to swap out the file that defined the formatting and then recompiled all CVs with the new template. A competent system administrator could have done this in a few minutes, yet instead this task needed to be done individually by thousands of people.

²It is possible to deviate from the template locally if you really want to but typically this is awkward to do. This is because you are trying to defy the compiler which almost always means you are attempting to commit crimes against typesetting and should not be doing whatever you are trying to do.

3 Technical Writing

3.1 Mathematics

One of the main powers of $\text{\LaTeX}$ is the ability to format mathematics. In equation (1) below is a simple example showing closure of the $SO(2)$ group, something that would be extremely painful to typeset in Word but is trivial to do in $\text{\LaTeX}$.

$$R(\theta)R(\phi) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix} \quad (1a)$$

$$= \begin{pmatrix} \cos \theta \cos \phi - \sin \theta \sin \phi & -\cos \theta \sin \phi - \sin \theta \cos \phi \\ \sin \theta \cos \phi + \cos \theta \sin \phi & -\sin \theta \sin \phi + \cos \theta \cos \phi \end{pmatrix} \quad (1b)$$

$$= \begin{pmatrix} \cos(\theta + \phi) & -\sin(\theta + \phi) \\ \sin(\theta + \phi) & \cos(\theta + \phi) \end{pmatrix} \quad (1c)$$

$$= R(\theta + \phi) \quad (1d)$$

We note the following that are impossible, awkward, or very manual to do in Word:

- The equations are numbered automatically and we do not need to position them ourselves.
- The equations are aligned on the equals sign - I did not need to add spaces to do this.
- The whole equation or individual parts such as equation (1b) can be cross-referenced easily.
- Much more complicated configurations can be created, such as equations in two columns.

3.2 Images

Images are inserted by giving a path to the image. This means that the document also updates when images are updated when it is recompiled. Copying by reference means there is one version of the truth which is one of the reasons why copying by value in situations like this is poor practice. I have needed to spend a lot of time reinserting images into documents because it appears Word can only copy by value.

Word and other Microsoft Office Suite products are very poor at handling vector graphics. As an engineering consultancy, we feature many images which could be vector graphics yet are forced to be rasterised, lowering their quality. The only option I have found that Word is happy with are Enhanced Metafiles (EMF), a format of Microsoft's creation with many problems³. On the other hand, $\text{\LaTeX}$ handles both bitmaps and vector graphics easily.

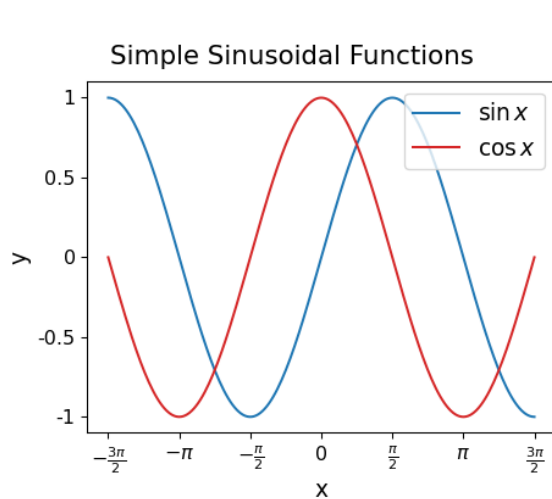
Word's strange behaviour with images and exporting wastes time. Word will compress images for you by default when pasting an image into the document, and even stranger, the level of compression depends on the zoom level of the editor. I have also seen documents where all the figures are unreadable because Word decided to compress them between revisions. There are many ways to export to PDF in Word, all of which behave differently and all of which compress images no matter what settings are used. $\text{\LaTeX}$ will produce a PDF with the exact images it was given.

3.3 Plots and Diagrams

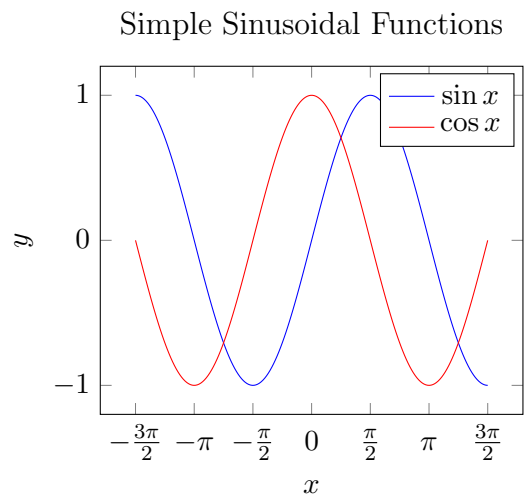
Going further than this, plots and figures can even be created inside $\text{\LaTeX}$ itself, although this is typically for more advanced users. This means the font in plot titles and axis labels can be consistent with the rest of the document and even the entire project. I have not found any other way of achieving this conveniently for figures that scale to the given size automatically. Figure 1 demonstrates the difference in quality. Even though figure 1a has been well-made in Python, the lower resolution, inconsistent style, and nasty typesetting of π symbols make it look noticeably worse than figure 1b. Figure 1b has infinite resolution and the added benefit of highlightable and searchable text⁴.

³EMF files do not support transparency. They also do not support antialiasing so all text looks odd as well.

⁴It is not necessary to make the figure in $\text{\LaTeX}$ for this functionality, any vector image would have this.



(a) 96 dpi raster made with matplotlib



(b) Vector graphic made in L^AT_EX

Figure 1: A comparison between a image rasterised with low-quality and a vector graphic.

3.4 Other

Below is a list of other features of L^AT_EX that make it better for writing technical documents that Word does not support.

- Subfigures can be given automatically numbered with easily cross-referencable subcaptions.
- Figures and tables can be given an optional short caption which appears in the list of tables and list of figures. This means brief metadata about figures can be kept closely attached to the figures and tables in the caption without cluttering the corresponding lists.
- The exact positioning of objects is handled automatically. For example, the user provides the caption text and the compiler puts it in a sensible place.
- All formatting changes can be tracked and easily copied.
- Smaller file sizes and the ability to handle documents several hundred pages long easily.
- The output is determined exactly from the text input so it is not possible to reload the output and get different behaviour like in Word.

4 Version Control

In this section I want to compare currently used version control methods for Word documents with what is possible with $\text{\LaTeX}$. One of the main powers of $\text{\LaTeX}$ is that it is fully compatible with text-based version control software such as Git, the free and open source software used by almost all developers and software companies. Here we will not look at all the features of Git or how it works, but trust that it is more than powerful enough for the needs of WSP.

We will look at an approach to version control used on my current project and then see what is possible with a combination of $\text{\LaTeX}$ and Git. There are two main types of differences that someone may care about tracking as given below.

- Changes in meaning such as “Updated system description to comply with requirement xyz”.
- All changes to the text and formatting.

4.1 Current Practice

On my current project we track the former type of change in a table of changes and the latter type of change with cross-through formatting and green highlighting. It may be the case that this project has a uniquely horrifying version control policy, but nonetheless it is worth highlighting the problems with it.

- **Manually recording changes.** If someone forgets to record the change or incorrectly applies the formatting then the change is not properly tracked. Git does this automatically it saves the need for the author to do it and also ensures it is always done correctly.
- **The document is harder to read.** The current version should not be polluted by the remnants of superseded parts. I regularly see paragraphs weaving in and out of superseded and non-superseded text which makes the current version hard to parse. It is not possible to easily see the document without the version control formatting either.
- **Updating revisions.** When a new revision is being prepared, the first task is to remove all text with cross-through formatting and remove all green highlight. This is a manual task that should not exist. As before, this process is also error prone - consider figure 2a where it is almost impossible to see the difference between a four that has been removed from the document and one that has been added. Figure 2b has a single row that has been removed that could easily be missed when preparing the next revision.
- **Formatting.** Formatting in Word is often inherent to the subject of the formatting or is given by invisible characters⁵ so cannot be tracked by this method. Other changes in formatting such as the structure of a table can only be recorded by duplicating the whole table and applying cross-through formatting.
- **Confusion between types of change.** There is no distinction between the two types of changes, often making for useless tables of changes. I have seen sections completely rewritten with the description “Text revised”, and the tense of a single unimportant word changed where the letters that were added and removed were stated. Both of these examples followed the version control policy exactly.

4.2 Resolution with $\text{\LaTeX}$ and Git

Before we look into how Git can avoid these issues we must first have some understanding of Git. In short, after some changes are made the author creates something called a “commit” which is a snapshot of the project at that point in time⁶, and then this commit can be sent to the remote repository. [This video](#) gives a simple explanation of the Git workflow for those wanting to know more.

⁵For example, the formatting of a bullet point or paragraph number is given by an invisible formatting character at the end of the clause and I have seen multiple people confused as to why they cannot clear this formatting.

⁶Git does not store only the changes and it also does not create a copy of the whole project. It works in a clever way that is beyond the scope of this essay that means the total amount of stored data is small for sensible use cases.



Smoke and Heat Control Systems	BS 12101 Part 3 2015
Part 3: Specification for powered smoke and heat control ventilators (Fans)	
PRAM Management Plan	1CP02-BVS_WSP-EN-PLN-SS07-000001
PRAMS Demonstration and Acceptance Report (SP1_01)	1CP02-BVS_WSP-EN-REP-SS07-000004
WP10a Roof Package, Architectural Drawings	1CP02-BVS_WSP-AR-DPP-SS07-624300 series
WP11a Façade Package, Architectural Drawings	1CP02-BVS_WSP-AR-XXX-SS07-623000 series
Architectural Specification (SP1_01)	1CP02-BVS_WSP-AR-SPE-SS07-000001.
Louvre Schedule (SP1_01)	1CP02-BVS_WSP-AR-SCH-SS07-000006
Facade Performance Specification	1CP02-BVS_WSP-ST-SPE-SS07-000001
WP23-WP24-WP27-WP28-MEPH-Comms and Information Fire Detection and Fire Suppression IDC Design Review Pack (SP1_01)	1CP02-BVS_WSP-AU-REV-SS07-000037
Acoustic Specification - Sound Insulation	1CP02-BVS_WSP-EV-SPE-SS07-000005
Interface Control Document – HRS02 M&E and Tunnel Ventilation	HS2-HS2-IN-ICD-000-000014; P02
HS2 Station Ventilation Platform Ventilation Passive Provision Schedule (SP1_01)	1CP02-BVS_WSP-RV-SCH-SS07-000001

(a) Strike-through comparison

(b) Easy to miss where text is removed

Figure 2: Examples of a current method of version control for Word documents.

The difference between any two commits can be produced on command and an example is seen in figure 3. While the version control process described above only gives the differences between neighbouring versions, Git can produce the difference between any two versions. As formatting is defined with text, such changes are easily tracked using this method as well. Git has a variety of tools for querying how two entities are different to each so things like whether any of the inserted images have changed or which files have changed, moved, or been deleted can be found easily as well.

There is even a tool specically for the $\text{\LaTeX}$ and Git workflow to take these identified differences and produce a PDF showing them. This is demonstrated in figure 4 which can be produced at any moment with a single simple command. This recreates the result of the previously outlined method but without any of the drawbacks of needing to format changes manually.

75 Below is a list of other features of $\text{\LaTeX}$ that make it better for writing technical documents that Word does not support.	80 Below is a list of other features of $\text{\LaTeX}$ that make it better for writing technical documents that Word does not support.
76	81
77 $\backslash\text{begin}\{\text{itemize}\}$	82 $\backslash\text{begin}\{\text{itemize}\}$
78 $\backslash\text{item}$ Subfigures can be given automatically numbered and easily cross-referencable subcaptions.	83+ $\backslash\text{item}$ Subfigures can be given automatically numbered with easily cross-referencable subcaptions.
79 $\backslash\text{item}$ Figures and tables can be given an optional short caption which appears in the list of tables and list of figures. This means brief metadata	84 $\backslash\text{item}$ Figures and tables can be given an optional short caption which appears in the list of tables and list of figures. This means brief metadata about figures

Figure 3: An example of the difference between two files.

Each commit contains metadata about who made the commit. This means who did what change and when that change was made is also tracked. A commit is given a brief description of the change and includes a file where more detail can be provided. This means why each small change was made can also be tracked. I was working on a project where “Should” was changed to “Shall” and it caused a four-year long saga. This change should not have happened, and if the reason for the change was understood then perhaps it could have been reverted. While name and date information can be tracked using Word’s track changes feature, if someone simply accepts all the changes then this information is lost. In contrast, a commit is immutable so cannot be changed once created⁷.

⁷Git uses SHA-256 hashes to ensure this.

Images are inserted by giving a path to the image. This means ~~when the images are updated, that~~ the document also updates when images are updated when it is recompiled. Copying by reference means there is one version of the truth which is one of the reasons why copying by value in situations like this is poor practice. I have ~~wasted~~ needed to spend a lot of time reinserting images into documents because it appears Word ~~only copies~~ can only copy by value.

Word and other Microsoft Office Suite products are very poor at handling vector graphics. As an engineering consultancy, we feature many images which could be vector graphics yet are forced to ~~rasterise~~ be rasterised, lowering their quality. The only option I have found that Word is happy with are Enhanced Metafiles (EMF), a format of Microsoft's creation with many problems². On the other hand, L^AT_EX handles both bitmaps and vector graphics easily.

Word's strange behaviour with images and exporting wastes time. Word will compress images for you by default when pasting an image into the document, and even stranger, the level of compression depends on ~~how zoomed in you are~~ the zoom level of the editor. I have also seen documents where all the figures are unreadable because Word decided to compress them ~~all~~ between revisions. There are many ways to export to PDF in Word, all of which behave differently and ~~compress images~~ all of which compress images no matter what settings are used. L^AT_EX will produce a PDF with the exact ~~image~~ images it was given.

Figure 4: An extract from an automatically generated PDF showing differences between versions.

L^AT_EX and Git are also good at maintaining a single source of truth. When a duplicate copy of something is made it is almost certain that they will diverge, making future interpretation of which is true a nightmare. This is why copying by value should be avoided unless absolutely necessary. As previously mentioned, L^AT_EX inserts figures and other documents by reference. In Git there is only one remote repository that everything ultimately syncs with. Compare this with Word where figures and other included documents are necessarily copied by value and where the file can be downloaded or otherwise copied. Passing comment sheets around via email is also an easy way to end up with muddled versions.

4.3 Comments and High-Level Changes

Comments about a document can be discussed by raising an issue on the remote repository. In figure 5 I have simulated this process where I act as both a client and the author. Comments can be added to a document. Details of what was changed in a commit can be written

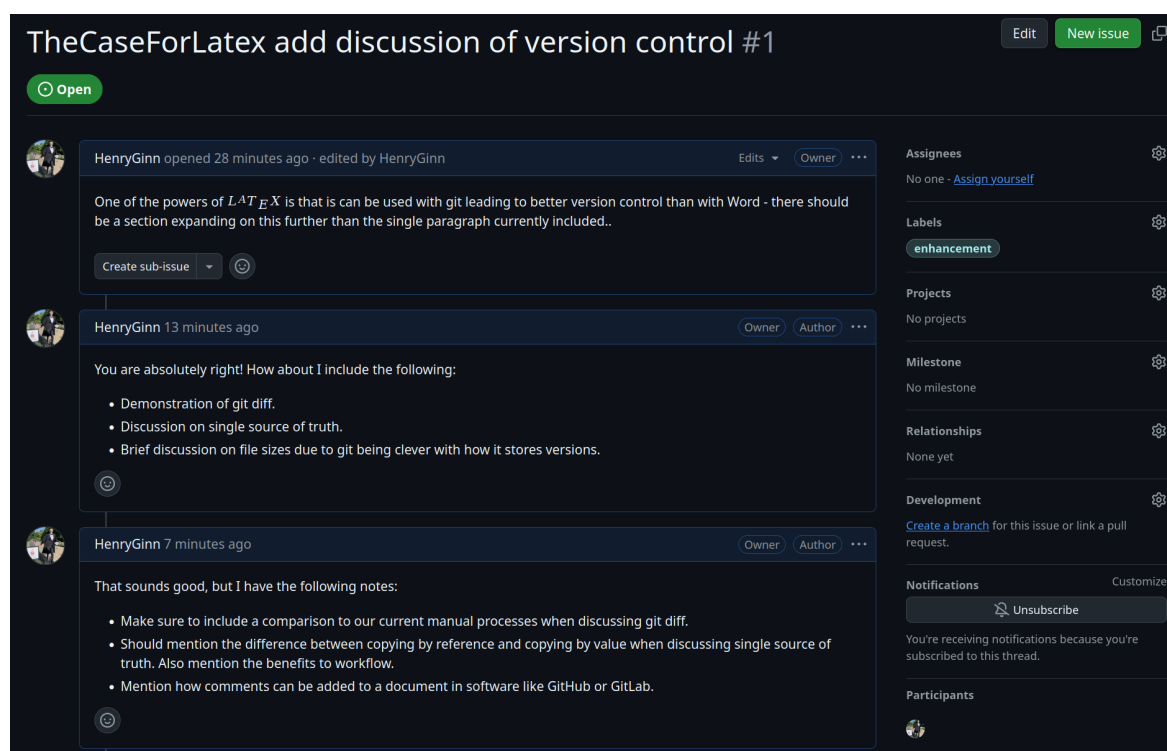


Figure 5: How comments can be made on a document in GitHub.

4.4 Other Features of Git

Almost all the power of Git will not be needed by almost all users and they can likely work just fine with a very simple workflow interacting with Git via a graphical user interface. There are more advanced features of Git that are worthy of note however.

- **Branching.** Branching allows multiple versions of a document to be worked on concurrently. This is useful when a different aims or directions are being explored.
- **Hashing documents.** Git stores a hash of documents meaning whether they have changed can be verified. This is useful when PDF files are attached as appendices.
- **Signing commits.** If the undisputable time and source of a commit is necessary, it is possible to do this with integration with GNU Privacy Guard. This is cryptographically secure and allows the validity of commits to be verified⁸.

4.5 Implementation

A host for any remote repositories is needed. Microsoft owns GitHub and provides hosting services, but there are also other options such as GitLab which is free and open source. This means it can be self-hosted by WSP, removing dependence on external parties and keeping all uploaded files completely internal to WSP.

While the use of Git is standard in software engineering, it may be deemed too complex for all employees to utilise effectively. Real-time collaborative editing can be done using Overleaf which is free and open source software. As before, this means WSP is able to self-host it, with all the independence and security benefits as before.

⁸For good reasons outside the scope of this essay, Git uses a user's system time (or even a time given explicitly given by the user) in a commit meaning the time can be spoofed without this method.

5 Ease of Use

5.1 Overview

The cost of these improvements is that $\text{\LaTeX}$ is less simple to use than Word, although I would argue the gap is not that large, especially for basic word processing. Table 1 summarises how complex certain tasks are to do in $\text{\LaTeX}$ with examples given below. The general principle of simple aims being simple to execute and complex aims being complex to execute holds, as long as it is inline with good typesetting principles. For example, writing text sideways might need some research.

Table 1: The experience and training needs to do certain tasks in $\text{\LaTeX}$.

Task	Difficulty
Typing in paragraphs	As easy as in notepad.
Creating headings	Use the <code>\section{}</code> , <code>subsection{}</code> , and <code>subsubsection{}</code> commands instead of applying a style.
Cross-referencing	Use <code>\label{label name}</code> in the relevant location and then <code>\ref{label name}</code> to reference that location.
Figures and tables	Requires some code which can be copy and pasted. For basic cases the formats are easy to extend from examples.
Single equations	Nearly as easy as Word, for example <code>\alpha</code> for α .
Complex equations	Will need some practice and around an hour of training.
Making a template	Intermediate level of difficulty, but only needs to be done once per project. Regular users do not need to be at this level.
Making diagrams	Intermediate to advanced, but unnecessary for what we do at WSP and can be replaced by a figure produced in other software anyway.

5.2 Most Complicated Example

Inserting images is likely the most difficult task most employees would need to do. The below code produces two subfigures side by side, although could easily be modified to have three. I posit that most WSP employees could deduce that they would need to add another subfigure block and lower the width.

```
\begin{figure}[H]
  \begin{subfigure}{0.45\textwidth}
    \includegraphics[width=\textwidth]{example-image-a}
    \caption{Example A}
    \label{Example Subfigure 1}
  \end{subfigure}
  \hfill
  \begin{subfigure}{0.45\textwidth}
    \includegraphics[width=\textwidth]{example-image-b}
    \caption{Example B}
    \label{Example Subfigure 2}
  \end{subfigure}
  \caption[Short caption that appears in list of figures]{Overall figure caption}
  \label{Overall Figure Caption}
\end{figure}
```

$\text{\LaTeX}$ has been around for around 40 years and is the standard in most technical fields in academia. Common packages are well-documented and there is a large repository of questions and answers on the dedicated $\text{\TeX}$ Stack Exchange. When an error occurs, a benefit of $\text{\LaTeX}$ over Word is that the issue is almost always due to the user doing something wrong rather than strange behaviour of $\text{\LaTeX}$. This means the user has control over the problem and is able to fix it.

6 Failures of Word

In this section I want to give some examples of times where the use of Word has caused me significant grief which would never have been a problem when using a combination of L^AT_EX and Git. Here I want to highlight how much time and energy is wasted by using Word. The section is concluded by a list of annoying behaviours of Word.

6.1 Attaching PDF Documents in Appendices

Several times I have had to attach spreadsheets as an appendix to a PDF as part of a deliverable⁹. Below is the process we use to get the correct page numbers in the footer and table of contents and links that go to the intended page:

1. Go to the first sheet of the spreadsheet being included, change the “start at page number” setting to match the position it belongs in the document, and export it.
2. Repeat this for the next sheets of the spreadsheet, ensuring that the page number adds the length of the previous sheets that have been converted to PDF.
3. After the corresponding appendix title page, add blank pages equal to the length of the PDF being included.
4. Repeat for all spreadsheets being included (or just step 3 for existing PDFs).
5. After exporting Word to PDF, remove the blank pages in a pdf editor and add in the pages of the PDFs to be included.

There are several problems with this approach. Firstly, this is an error-prone process. The chances of pages being added in the wrong order is high and there are many opportunities to end up with incorrect page numbers and table of contents links anyway. Secondly, if any changes to the document are made that change the length of the document then the whole process will need to be redone. Finally, this process can take a long time. I once complained to a colleague how I had ended up wasting a whole day converting a document to a PDF and they told me they had done the same thing before.

Let us consider how this works in L^AT_EX. All sheets are exported to PDF without page numbers and there is a simple single line of code that tells the compiler to include a PDF with page numbers. All links and page numbers are done for you and are updated automatically in response to any changes. In addition to this, these page numbers (and any other header or footer information) will follow the document template given and would not need to be recreated in Excel.

6.2 Version Control

Recently I had a Sharepoint synced folder that stopped syncing and I did not notice for a month. This meant that several documents I was working on had diverged from the main version and I would need to merge any changes I had made into the main document. To be fully confident I had not missed anything, I needed to copy both documents into Notepad, remove the clause numbers to stop superfluous differences being detected, and then use a tool that identified differences between two texts. After this I had to check all formatting was correct because this method would not have been picked up formatting differences.

I ended up wasting a whole day on this task. As described in section 4, this was something that Git was designed for and would therefore have taken only a few minutes per document.

6.3 Changing Template

Recently a colleague had used a WSP template for some work where it was subsequently decided that it belonged in a document for the project, meaning changing to the project template. No matter what type of paste special I used, the styles of the project template would not apply and I needed to go

⁹Personally I believe this is not best practice and we should be sending the native file, but we will ignore that.

through and apply the styles by hand. This was not too arduous for a 12-page document, but there were many tables that needed formatting. This should have been a case of applying the table header style to the header and the table body style to the body, but that did not work. Instead the font size, font type, font colour, font alignment, cell borders, cell alignment, and paragraph alignment had to be set manually, for both the header and the body for every table¹⁰.

I spent an afternoon fixing this. Using L^AT_EX this would have been solved by a single copy and paste because the formatting is handled elsewhere. Worse, it turned out that my colleague had a more up-to-date version of the document stored locally, so I had to carefully scan for differences between the tables and update them manually. If using L^AT_EX then I could have copy and pasted again.

6.4 Assorted Complaints of Word

In case it is not clear how unstable, buggy, and annoying Word is, I have included a few oddities and frustrations of Word below:

- Tables split across page breaks by default.
- Captions are not attached to the corresponding object by default.
- Sometimes invisible, unselectable, and undeletable formatting characters can appear in your document that break your formatting. The only fix for this is awkwardly manoeuvring the character somewhere it does no harm or copying everything into a new document.
- Clicking the bullet point button can change the font size and font style.
- Space after paragraph should be used to ensure space between paragraphs, but the space is still added for the last paragraph on the page. As the space is attached to the paragraph, it puts the line on the next page even if that line would fit above. To avoid widows, space after paragraph needs to be manually handled.
- The gap between “Figure” or “Table” and the number in the caption is not a non-breaking space by default. This can result in the number being on a new line when cross-referencing.
- Vertically centred text in a cell of a table does not appear vertically centred, even if space before and after paragraph are the same and margins are symmetrical.
- Page breaks take up space. This means if the last line that fits on the page is long then the page break mark is moved to the next page, meaning a blank page. This means needing to manually handle the page break. This often occurs with tables and figures.
- Changing the width of a column in a table where the last cell highlighted is merged will only change the width of the first row of the merged cell. This means needing to change the width of those cells separately and remerging afterwards.
- The number of words that fit on a line depends on their justification - this should only affect the spacing between words (unless rivers are automatically avoided which Word cannot do).
- The default behaviour of images is “keep with next”, yet if two images will not fit on the same page together then word will put them on separate pages with an automatic page break on the preceding page, even if there was room for the first image¹¹.

¹⁰This template uses a different font style and size for the header than the rest of the document. This would be awkward to do in L^AT_EX because it is terrible typesetting practice.

¹¹If two images are meant to be understood together then they should be subfigures of a single figure object.

7 Conclusion

In summary, it is far faster and easier to produce technical documents in $\text{\LaTeX}$ once familiarity has been attained, and the results are of higher quality and appear more professional. Table 2 below gives a comparison of the key differences between Word and $\text{\LaTeX}$. The source code for this essay is available on my [GitHub](#).

Table 2: Comparing Word and $\text{\LaTeX}$.

Property	Word	$\text{\LaTeX}$
Editor format	What You See Is What You Get	Needs to compile
Bugfixing	Poor support and confusing behaviour	Good support with searchable errors
Software	Proprietary	Free and Open Source
Formatting control	Poor	High
Technical features	Lacking	Rich
Microsoft compatibility	High	Medium
Simplicity	Extremely simple	$\sim$ 1 hour of training
Version control	Medium	Git-compatible

$\text{\LaTeX}$ is standard software in academia. Git is standard software in software engineering. They are both extremely mature and have been successfully implemented in these sectors so I see no reason why this could not also be the case in engineering and consultancy. Collaboration, version control, and producing technical documents is a hard problem, but it has been solved by $\text{\LaTeX}$ and Git. Therefore I hope that WSP agrees that their use is inline with their digital approach and considers implementing this technology.